

Series 4: Applied Machine Learning and Predictive Modelling 1

Dr. Luisa Barbanti and Dr. Matteo Tanadini

Fall Semester 2025 (HSLU)

*** Solutions ***

Load the `x-ray` dataset (file “Xray.RDS”). This study refers to a “study carried out to investigate the connection between X-ray usage and acute myeloid leukemia in childhood” (for further information see the help page about the `amlxray` dataset in `{faraway}` package). We work on a subset of the original data, in particular, we consider the following variables:

- `disease`: is the response variable (1 = *diseased*, 0 = *healthy*)
- `Sex`: Sex of the patient
- `age`: Age of the child (in years)
- `downs`: Presence of the Down syndrome
- `Mray`: Did the mother ever have an Xray
- `Fray`: Did the father ever have an Xray
- `CnRay.num`: Total number of Xrays of the child "1" = none; "2" = 1 or 2; "3" = 3 or 4; "4" = 5 or more

```
d.xray <- readRDS("../.../Datasets/Xray.RDS")
str(d.xray)
```

```
Classes 'tbl_df', 'tbl' and 'data.frame':  112 obs. of  7 variables:
 $ Sex      : Factor w/ 2 levels "F","M": 1 2 1 2 2 1 1 2 1 2 ...
 $ disease  : num  1 1 0 1 1 1 1 0 0 0 ...
 $ age      : int  0 6 8 1 4 9 17 5 0 7 ...
 $ downs    : Factor w/ 2 levels "no","yes": 1 1 1 2 1 1 1 1 1 1 ...
 $ Mray     : Factor w/ 2 levels "no","yes": 1 1 1 1 2 2 1 1 1 1 ...
 $ Fray     : Factor w/ 2 levels "no","yes": 1 1 1 1 1 1 1 1 1 1 ...
 $ CnRay.num: num  1 3 1 1 1 1 2 1 1 1 ...
```

```
head(d.xray)
```

	Sex	disease	age	downs	Mray	Fray	CnRay.num
1	F	1	0	no	no	no	1
2	M	1	6	no	no	no	3
3	F	0	8	no	no	no	1
4	M	1	1	yes	no	no	1
5	M	1	4	no	yes	no	1
6	F	1	9	no	yes	no	1

The response variable is *disease*, which is a binary variable. We will not use *downs* as predictor, but all the other variables can be included as predictors in your models.

The goal is to use 5-fold cross validation to compare four models. As a measure of predictive performance we want you to use the proportion of correctly classified observations for the test sample. The following questions will sequentially build the implementation of this goal.

Question

Fit the following four models on the whole dataset. Take a look at the summary-output of the four models. Compute the in-sample AIC-values (= AIC-values of the models fitted on the whole dataset) of all four models. You can use the function `AIC()`. Based on these in-sample AIC-values, which model would you select as the best performing model?

```
glm.xray.simple <- glm(formula = disease ~ age + Sex,
                      data = d.xray,
                      family = "binomial")

##
glm.xray.complex <- glm(formula = disease ~ age + Sex + Mray + Fray + CnRay.num,
                      data = d.xray,
                      family = "binomial")

##
glm.xray.very.complex <- glm(formula = disease ~ Sex * (poly(age, degree = 2) +
                      Mray + Fray + CnRay.num) ,
                      data = d.xray,
                      family = "binomial")

## we need to load the package mgcv to use the function gam()
library(mgcv)
gam.xray <- gam(formula = disease ~ s(age, by = Sex) + Sex + Mray + Fray + CnRay.num,
               data = d.xray,
               family = "binomial")
```

Answer

```
glm.xray.simple <- glm(formula = disease ~ age + Sex,
                      data = d.xray,
                      family = "binomial")

##
summary(glm.xray.simple)
```

Call:

```
glm(formula = disease ~ age + Sex, family = "binomial", data = d.xray)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	-1.2364	0.3954	-3.13	0.0018 **
age	0.0653	0.0382	1.71	0.0875 .
SexM	0.9149	0.4008	2.28	0.0225 *

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 152.97 on 111 degrees of freedom
Residual deviance: 143.63 on 109 degrees of freedom
AIC: 149.6

Number of Fisher Scoring iterations: 4

```
##  
glm.xray.complex <- glm(formula = disease ~ age + Sex + Mray + Fray + CnRay.num,  
                        data = d.xray,  
                        family = "binomial")  
##  
summary(glm.xray.complex)
```

Call:

```
glm(formula = disease ~ age + Sex + Mray + Fray + CnRay.num,  
    family = "binomial", data = d.xray)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	-2.0239	0.5447	-3.72	0.0002 ***
age	0.0391	0.0406	0.96	0.3352
SexM	0.9348	0.4164	2.24	0.0248 *
Mrayyes	0.6685	0.7722	0.87	0.3866
Frayyes	0.1062	0.4382	0.24	0.8085
CnRay.num	0.6494	0.3114	2.09	0.0370 *

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 152.97 on 111 degrees of freedom
Residual deviance: 137.76 on 106 degrees of freedom
AIC: 149.8

Number of Fisher Scoring iterations: 4

```
##  
glm.xray.very.complex <- glm(formula = disease ~ Sex * (poly(age, degree = 2) +  
                           Mray + Fray + CnRay.num) ,  
                           data = d.xray,  
                           family = "binomial")  
##  
summary(glm.xray.very.complex)
```

Call:

```
glm(formula = disease ~ Sex * (poly(age, degree = 2) + Mray +  
    Fray + CnRay.num), family = "binomial", data = d.xray)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	-2.256	0.869	-2.60	0.0094 **
SexM	1.517	1.075	1.41	0.1584
poly(age, degree = 2)1	0.891	4.089	0.22	0.8276
poly(age, degree = 2)2	-1.551	3.242	-0.48	0.6323
Mrayyes	0.537	1.040	0.52	0.6054
Frayyes	-0.760	0.799	-0.95	0.3413
CnRay.num	1.191	0.623	1.91	0.0560 .
SexM:poly(age, degree = 2)1	1.386	5.009	0.28	0.7821
SexM:poly(age, degree = 2)2	0.730	4.509	0.16	0.8714
SexM:Mrayyes	0.139	1.668	0.08	0.9337
SexM:Frayyes	1.319	0.988	1.33	0.1822
SexM:CnRay.num	-0.724	0.725	-1.00	0.3184

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 152.97 on 111 degrees of freedom
Residual deviance: 134.69 on 100 degrees of freedom
AIC: 158.7

Number of Fisher Scoring iterations: 4

```
## we need to load the package mgcv to use the function gam()  
library(mgcv)  
gam.xray <- gam(formula = disease ~ s(age, by = Sex) + Sex + Mray + Fray + CnRay.num,  
    data = d.xray,  
    family = "binomial")  
##  
summary(gam.xray)
```

Family: binomial

Link function: logit

Formula:

disease ~ s(age, by = Sex) + Sex + Mray + Fray + CnRay.num

Parametric coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	-2.357	0.663	-3.56	0.00038 ***
SexM	1.236	0.476	2.60	0.00939 **
Mrayyes	0.773	0.852	0.91	0.36451
Frayyes	0.308	0.489	0.63	0.52860
CnRay.num	0.878	0.387	2.27	0.02348 *

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Approximate significance of smooth terms:

	edf	Ref.df	Chi.sq	p-value
s(age):SexF	6.19	7.34	9.47	0.24
s(age):SexM	5.87	7.06	6.79	0.45

R-sq.(adj) = 0.187 Deviance explained = 25%
UBRE = 0.32964 Scale est. = 1 n = 112

We now recompute the in-sample AIC for the glm-models (note that you also find the AIC-value in the summary-outputs for glms) and also compute it for the gam-model.

```
AIC(glm.xray.simple)
```

```
[1] 150
```

```
AIC(glm.xray.complex)
```

```
[1] 150
```

```
AIC(glm.xray.very.complex)
```

```
[1] 159
```

```
AIC(gam.xray)
```

```
[1] 149
```

Based on this in-sample AIC, we would choose the gam-model as the best performing one. However, its AIC-value is almost the same as the AIC-value of the glm.simple and the glm.complex.

Question

Calculate the proportion of correctly classified observations in-sample (that is on the whole data set as well) for all four models. Use a cutoff of 0.5 in the predicted probability for disease. This means that if a patient has a predicted probability of 0.5 or higher, we would predict disease = 1 for this patient. Based on the in-sample proportion of correctly classified observations, which model would you choose?

Hint: Use type = “response” when predicting from a glm/gam with binary response, to obtain the predicted probabilities. You can write a function that has as two arguments the “model” and the “newdata” to compute the proportion of correctly classified observations. We can then reuse this function later.

Answer

We write a function that computes the proportion of correctly classified observations on a given input sample for a given input model. This function first computes the predicted probabilities for each observation in the input sample and then computes the proportion of correctly classified observations. We will use this function again later.

```
prop.correct.classified <- function(model, newdata){
  pred.proBABILITIES <- predict(object = model,
                                newdata = newdata,
                                type = "response")
  pred.disease <- ifelse(pred.proBABILITIES >= 0.5, 1, 0)
  mean(pred.disease == newdata$disease)
}
prop.correct.classified(model = glm.xray.simple, newdata = d.xray)
```

```
[1] 0.62
```

```
prop.correct.classified(model = glm.xray.complex, newdata = d.xray)
```

```
[1] 0.68
```

```
prop.correct.classified(model = glm.xray.very.complex, newdata = d.xray)
```

```
[1] 0.68
```

```
prop.correct.classified(model = gam.xray, newdata = d.xray)
```

```
[1] 0.75
```

Based on the in-sample proportion of correctly classified observations, we would again choose the gam model as best performing model.

Question

Now implement 5-fold CV to compare the estimated proportion of correctly classified observations on test data. If you want, you can implement repeated 5-fold CV to see how much the estimated proportion of correctly classified observations varies between different 5-fold CVs (different permutation of the rows of the data).

Answer

```
library(mgcv)
set.seed(121)
##
## 1) prepare data
## create 5 (roughly) equally sized folds:
## K = number of folds
K <- 5
n <- nrow(d.xray)
folds <- cut(1:n, breaks = K, labels = FALSE)
##
```

```

## perform K fold cross validation:
prop.correct.classified.glm.simple <- c()
prop.correct.classified.glm.complex <- c()
prop.correct.classified.glm.very.complex <- c()
prop.correct.classified.gam <- c()
## loop over test folds
for(k in 1:K){
  ## take the Kth fold as test set and the other folds as training set
  ind.test <- which(folds == k)
  d.xray.test <- d.xray[ind.test, ]
  d.xray.train <- d.xray[-ind.test, ]
  ##
  ## glm simple ##
  ##
  ## 2) fit the model with "train" data
  glm.xray.simple <- glm(formula = disease ~ age + Sex,
                        data = d.xray.train,
                        family = "binomial")
  ##
  ## 3) proportion of correctly classified observations for the test sample
  prop.correct.classified.glm.simple[k] <- prop.correct.classified(model = glm.xray.simple,
                                                                    newdata = d.xray.test)

  ## glm complex ##
  ##
  ## 2) fit the model with "train" data
  glm.xray.complex <- glm(formula = disease ~ age + Sex + Mray + Fray + CnRay.num,
                        data = d.xray.train,
                        family = "binomial")
  ## 3) proportion of correctly classified observations for the test sample
  prop.correct.classified.glm.complex[k] <- prop.correct.classified(model = glm.xray.complex,
                                                                    newdata = d.xray.test)

  ## glm very complex ##
  ##
  ## 2) fit the model with "train" data
  glm.xray.very.complex <- glm(formula = disease ~ Sex * (poly(age, degree = 2) +
                                                                Mray + Fray + CnRay.num),
                        data = d.xray.train,
                        family = "binomial")
  ## 3) proportion of correctly classified observations for the test sample
  prop.correct.classified.glm.very.complex[k] <- prop.correct.classified(model =
                                                                    glm.xray.very.complex,
                                                                    newdata = d.xray.test)

  ## gam ##
  ##
  ## 2) fit the model with "train" data
  gam.xray <- gam(formula = disease ~ s(age, by = Sex) + Sex + Mray + Fray + CnRay.num,
                 data = d.xray.train,
                 family = "binomial")

  ## 3) proportion of correctly classified observations for the test sample

```

```

prop.correct.classified.gam[k] <- prop.correct.classified(model = gam.xray, newdata =
                                                    d.xray.test)

}

## we obtain the estimated test proportion of correctly classified observations
## for the 5-fold CV by averaging over all 5 folds
( prop.correct.classified.glm.simple <- mean(prop.correct.classified.glm.simple) )

[1] 0.65

( prop.correct.classified.glm.complex <- mean(prop.correct.classified.glm.complex) )

[1] 0.6

( prop.correct.classified.glm.very.complex <- mean(prop.correct.classified.glm.very.complex) )

[1] 0.56

( prop.correct.classified.gam <- mean(prop.correct.classified.gam) )

[1] 0.57

```

Question

Based on the results of the 5-fold CV, which of the four models would you choose and why?

Answer

Note that we only applied 5-fold CV once, that is we did not apply it repeatedly on permuted observations, and our estimate of the proportion of correctly classified observations on test samples thus involves randomness. Note that this randomness is not visible in the code, since we created the 5 folds with the `cut()`-function, which always allocates a group of succeeding observations into the same fold (fold 1 consists of the first 5th of the data, fold two of the second 5h of the data and so on). Of course, this is still a random allocation of observations into folds.

According to our analysis, the best estimated test performance is obtained by the `glm.simple` model, which is the one we might choose based on this analysis. Recall that this model performed worst in-sample. In-sample, the best performing model was the `gam`-model (with respect to in-sample AIC and proportion of correctly classified observations). Thus, when estimating the performance of a model on unseen test data, the ranking among models can change.

Question

Ask generative AI about the evaluation of the predictive performance of models and critically compare the obtained answer with what you learned in this class.